

Recognition and construction of PP

Recognition:

	1	2	3	4	5
A	0	1	1	0	1
B	0	0	0	1	0
C	1	1	1	0	0
D	0	0	0	1	0
E	0	0	1	0	0

Input: Character State Matrix

1. Check the inclusion / disjointness property

a) Sort the columns decreasing by their number of '1's

	3	2	4	1	5
A	1	1	0	0	1
B	0	0	1	0	0
C	1	1	0	1	0
D	0	0	1	0	0
E	1	0	0	0	0
# 1s	3	2	2	1	1

b) Add a column number 0 containing only '1's

	0	3	2	4	1	5
A	1	1	1	0	0	1
B	1	0	0	1	0	0
C	1	1	1	0	1	0
D	1	0	0	1	0	0
E	1	1	0	0	0	0
# 1s	5	3	2	2	1	1

c) For each '1' in the sorted matrix, draw a pointer to the previous '1' in the same row

	0	3	2	4	1	5
A	1	1	1	0	0	1
B	1	0	0	1	0	0
C	1	1	1	0	1	0
D	1	0	0	1	0	0
E	1	1	0	0	0	0
# 1s	5	3	2	2	1	1

Conclusion: Do all pointers end in the column 0?

Yes

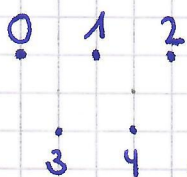
PP exists

No

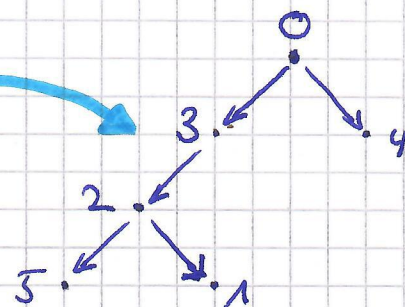
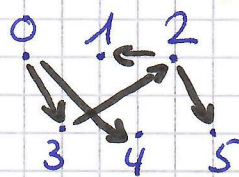
PP does not exist

Construction: 2. Build the phylogenetic tree

a) Create a graph with a node for each character

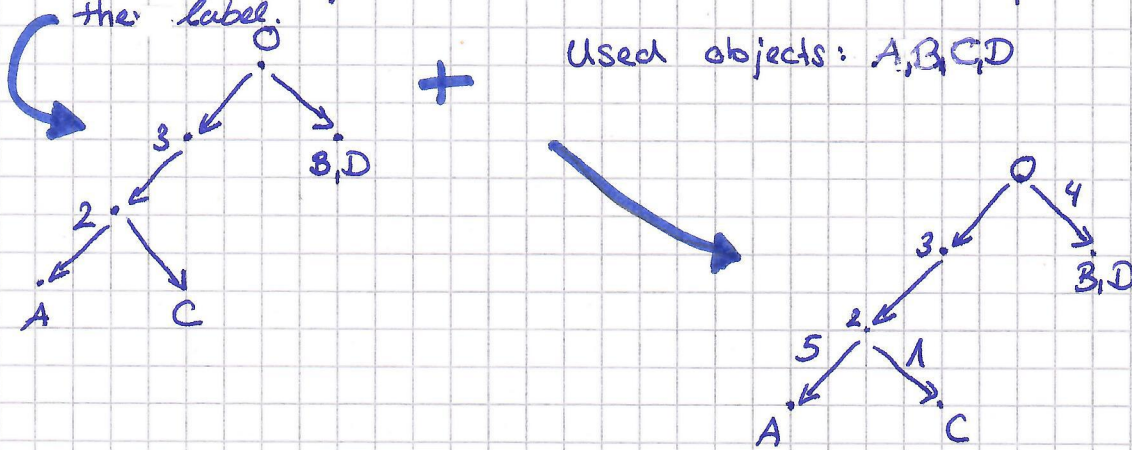


b) Add an edge between direct successors



3. Refine and annotate the tree

- a) Annotate all leaf nodes with their respective object sets, mark the objects as "used" and annotate the parent edge with the label.



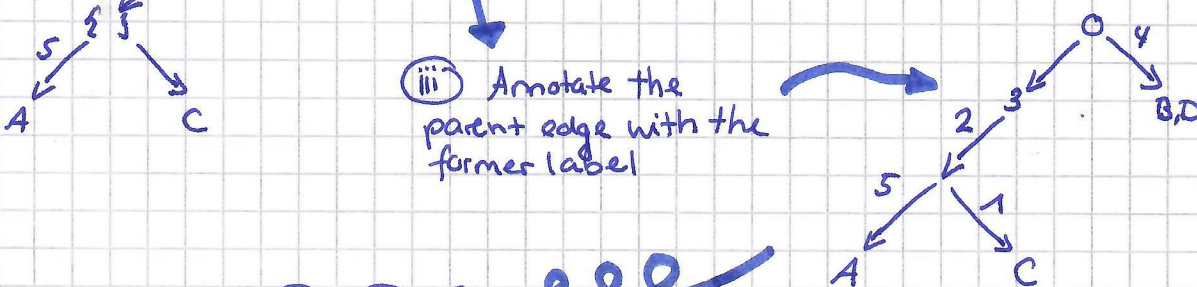
- b) For each internal node whose children are all re-labeled, do

(i) Re-label the node with all objects of its object set which are not yet marked as "used"

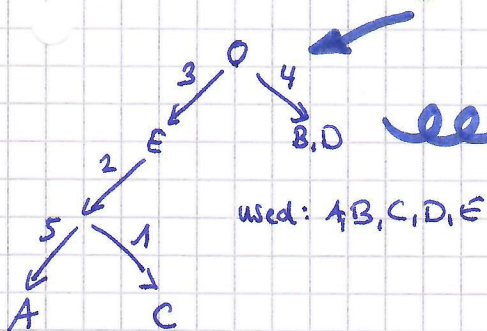
(ii) mark these objects as "used"

Used objects: A, B, C, D

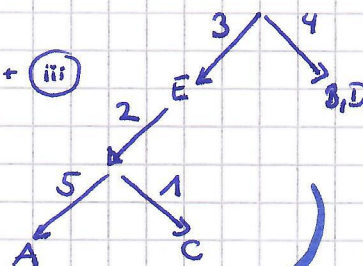
(iii) Annotate the parent edge with the former label



(i) + (ii) + (iii)



(i) + (ii) + (iii)



- c) If an internal node is labeled with an object, create a new branch from that node and annotate the newly created leaf with the object

